

B.M.S COLLEGE OF ENGINEERING

Bull Temple Road, Basavanagudi, Bangalore - 560 019, Karnataka, India



MINI PROJECT

SYNOPSIS

ON

“ACCELERATION OF 2D-CONVOLUTION ON A RISC-V CORE”

Submitted partial fulfilment of the requirement for completion of
MINI PROJECT [23EC5PWMPR]

Submitted by

Ojas Kudari	1BM22EC154
Rohith Vishnu Achari	1BM22EC201
Sateesh Y Abbigeri	1BM22EC223
Supraja S	1BM22EC207

2024 – 2025

Under the guidance

Dr.Jeeru Dinesh Reddy

Assistant professor

Department of Electronics and Communication Engineering

CONTENTS:

ACCELERATION OF 2D-CONVOLUTION ON A RISC-V CORE.....	1
INTRODUCTION.....	1
1. CHALLENGES AND OPPORTUNITIES IN CO-DESIGN OF CONVOLUTIONS AND RISC-V VECTOR PROCESSORS.....	2
2. ALGORITHMS FOR EFFICIENT COMPUTATION OF CONVOLUTION	3
3. IMPLEMENTATION OF FPGA-BASED FFT CONVOLUTION	4
4. FAST ALGORITHMS FOR CONVOLUTIONAL NEURAL NETWORKS.....	5
5. DESIGN AND IMPLEMENTATION OF 2D CONVOLUTION ON X86/X64 PROCESSORS	6
PROBLEM STATEMENT	8
PROPOSED SOLUTION	8
BLOCK DIAGRAM	9
PROJECT EXECUTION STEPS.....	10
REFERENCES	11
GUIDE RECOMMENDATIONS:.....	12

ACCELERATION OF 2D-CONVOLUTION ON A RISC-V CORE

Area of application/Domain: Image Processing, Convolution Neural Networks (CNN) /FPGA

INTRODUCTION

Convolution is a very important operation in Signal processing, image processing, training of ML models, Convolution Neural Networks. Multiplication is a very expensive operation in FPGA ,2D convolution similarly is also a very exhaustive multiplication operation. Convolution is extensively employed in the field of Image processing operations like edge detection, contour outlining, applying blurs and filters to image,

Processing high definition (HD), or even 4K images, involves millions of pixels, which require significant computational power for even basic operations like filtering or edge detection. There are many algorithms which significantly increase speed and have their own drawbacks. In this project we are using PicoRV32, which is an efficient, highly customizable open-source RISC-V core; its low footprint saves much power and is very flexible through instructions a, making it perfect for a resource-constrained environment compatible with FPGA, with great support from the development community.

LITERATURE SURVEY

1. CHALLENGES AND OPPORTUNITIES IN CO-DESIGN OF CONVOLUTIONS AND RISC-V VECTOR PROCESSORS

Introduction:

Over the years RISC architecture has developed, and along with it the usage of 'V' extension or Vector Extension, which allows us to use and implement processing for large amounts of data through parallelism. In this paper the convolution operation is further improved by utilization of Winograd algorithm using "V" extension. Various Vector extensions like Intel x86 MMX, SSE, AVX, AVX2, and AVX512, ARM NEON. Presenting an effective technique to compute on Winograd kernels

Methodology:

This work focuses on optimizing the "Winograd algorithm" for 3×3 kernel convolutions with a stride of 1, leveraging its ability to reduce computational complexity. The approach involves transforming both input and filter data into matrices, performing element-wise multiplications, and transforming the results back to the original domain. To fully utilize the long vector lengths available on the RISC-VV architecture, an inter-tile parallelization scheme is applied across input/output channels, allowing for efficient vectorization. This enables the use of longer vector lengths, which significantly boosts performance. For other convolutional layers that cannot benefit from Winograd, the im2col+GEMM algorithm is used, with optimizations applied to efficiently handle larger vectors.

Conclusion:

This work optimized the Winograd algorithm for RISC-VV and demonstrated speedups of $1.08\times$ for YOLOv3 and $1.76\times$ for VGG16 compared to im2col+GEMM. Winograd excels with 3×3 kernels, reducing computational complexity, Winograd's applicability is limited to layers with 3×3 kernels and stride 1, while im2col+GEMM remains more effective for other layer configurations. Driving us to seek a more robust alternative for higher kernel sizes

2. ALGORITHMS FOR EFFICIENT COMPUTATION OF CONVOLUTION

Introduction:

Convolution is an operation crucial to signal and image processing. Basic convolution, given by the Convolution theorem, computes the integral product of two functions namely the input signal and the kernel. In basic convolution algorithms, at higher dimensions, each dimension increases the degree of the algorithm polynomial linearly leading to expensive computation and increased time complexity. Thus, at higher dimensions, the convolution is often decomposed into lower dimension kernels and convolved step by step to reduce time complexity and memory requirement.

Methodology:

Two distinct approaches are involved to reduce memory requirements of fast convolution. Tiling is the first approach and is implemented in spatial or time domain. This reduces the memory requirements but also subsequently increases the number of arithmetic operations when the input data is decomposed into smaller sections or parts. DIF or decomposition in frequency is the second approach that is implemented in architectures where the overhead of data transfer is critical and is implemented by FFT. A decomposition in time or DIT approach is also briefly discussed, but it is not as efficient as either of the other two approaches.

Conclusion:

This research paper discusses multiple well-known algorithms, approaches and implementations of convolution. The authors also elaborate on the drawbacks, disadvantages and possible improvements of the same.

3. IMPLEMENTATION OF FPGA-BASED FFT CONVOLUTION

Introduction:

FFT-based convolution is a highly efficient algorithm for performing linear convolution by transforming signals into the frequency domain using the Fast Fourier Transform (FFT). This method reduces computational complexity, making it ideal for real-time signal processing applications like filtering, image processing, and communications systems.

Methodology:

The FFT-based convolution utilizes the overlap-add method to efficiently process streaming data. In this approach, input segments of length (L) are zero-padded with $(M - 1)$ zeros to ensure proper alignment during transformation. These padded segments are then transformed using an (N) -point FFT, which efficiently converts the time-domain signals into their frequency-domain representations. The frequency response of the filter is subsequently multiplied by the FFT of each segment, allowing for effective filtering in the frequency domain. Following this multiplication, an Inverse FFT is applied to convert the filtered frequency data back into the time domain.

A modified radix-4 FFT architecture is employed. This architecture significantly reduces hardware resource usage by sharing resources between the FFT and Inverse FFT blocks. Additionally, complex multiplication is further optimized by reducing the number of real multiplications required from four to three. This reduction not only minimizes hardware demands but also enhances computational efficiency. By leveraging these innovations, the proposed approach achieves an efficient balance between resource usage and high performance.

Conclusion:

In this research, a modified radix-4 architecture for FFT convolution methods, achieving significant resource efficiency and performance improvements, with a latency of just $1.16 \mu\text{s}$ versus $5.3 \mu\text{s}$ for existing designs. This work lays the foundation for optimizing multiplier usage, offering a promising direction for future enhancements.

4. FAST ALGORITHMS FOR CONVOLUTIONAL NEURAL NETWORKS

Introduction:

Convolutions are widely employed in tasks like image recognition and object detection, often requiring significant computational resources. Two leading methods for improving computational efficiency in Convolution are the Fast Fourier Transform (FFT) and Winograd's minimal filtering algorithm. While FFT is efficient for large filters, Winograd's algorithm shines with smaller filter sizes, particularly the 3x3 filters used in many state-of-the-art (Convolutional neural networks) CNNs.

Methodology:

1. Fast Fourier Transform (FFT):

FFT reduces convolution's arithmetic complexity by transforming the spatial domain into the frequency domain. By utilizing the convolution theorem, convolution becomes pointwise multiplication in the frequency domain. FFT is especially advantageous when working with large filters, where it provides substantial speedups. However, for smaller filter sizes like 3x3, FFT's overhead from complex number operations and the necessity of large tile sizes reduces its effectiveness.

2. Winograd's Minimal Filtering:

Winograd's algorithm is designed to minimize the number of multiplications required for convolution. For example, a typical 2x2 output with a 3x3 filter using direct convolution requires 9 multiplications, while Winograd reduces this to 4 multiplications. This reduction in arithmetic complexity makes it highly efficient for CNN layers using small filters and batch sizes, where it can outperform FFT, particularly when memory constraints are significant.

Conclusion:

Both FFT and Winograd algorithms have their places in optimizing CNNs. For networks relying on small filters and requiring low-latency inference, Winograd's algorithm presents a clear computational advantage due to its minimal arithmetic complexity. In contrast, FFT remains a strong choice for larger filter sizes and batch operations where its frequency-domain approach

provides significant computational savings.

5. DESIGN AND IMPLEMENTATION OF 2D CONVOLUTION ON X86/X64 PROCESSORS

Introduction:

2D convolution is a key computational operation in image processing tasks such as edge detection, smoothing, filtering, and Convolutional Neural Networks (CNNs). It is computationally intensive and highly dependent on the underlying hardware architecture, with different optimizations for architecture like x86/x64, ARM, and RISC.

Methodology:

Three primary approaches for accelerating 2D convolution have been identified:

Direct Convolution Optimization: This approach optimizes the convolution process through vectorization (using AVX/SSE for parallel processing), parallelization (via multi-threading like OpenMP), and memory optimizations (such as register blocking and bit-width reduction). Libraries like Intel's IPP and MKL provide support for these optimizations.

Matrix-Matrix Multiplication (MMM)-Based Methods: This approach reformulates convolution as matrix multiplication, using transformations like im2col. Libraries such as MKL and BLAS are commonly employed. However, it introduces memory overhead and inefficient CPU resource usage due to data reshuffling.

FFT and Winograd-Based Convolution: These algorithms reduce the number of floating-point operations, making them useful for larger kernel sizes. Winograd is particularly effective for smaller kernels (like 3x3), while FFT excels with larger kernels. However, both suffer from memory inefficiencies and precision issues on modern CPUs.

Recent Advances in Direct Convolution Optimization

Manual Vectorization: Hand-written vectorized code, which is more efficient than compiler-generated versions.

Bit-Width Management: Reducing intermediate results to lower bit-widths (e.g., 16-bit or 8-bit) to improve performance.

Low-Level SIMD Intrinsics: Fine-tuned control over vectorization and memory management using AVX/SSE intrinsics, bypassing the limitations of compiler auto-vectorization.

Challenges include the lack of efficient 8-bit vector multiplication instructions, inefficient compiler optimization, and memory bandwidth limitations, particularly in MMM-based methods.

Comparison of Techniques

Direct Convolution vs. MMM: Direct convolution is faster for small kernels (3x3, 5x5) as it avoids data reshuffling, while MMM benefits larger kernels but incurs memory overhead.

FFT and Winograd: These methods reduce computational complexity but suffer from performance issues due to memory inefficiency.

Conclusion

Optimizations for x86/x64 processors have shown significant gains in performance and energy efficiency, yet similar optimizations on RISC architectures remain a promising area for future research.

PROBLEM STATEMENT

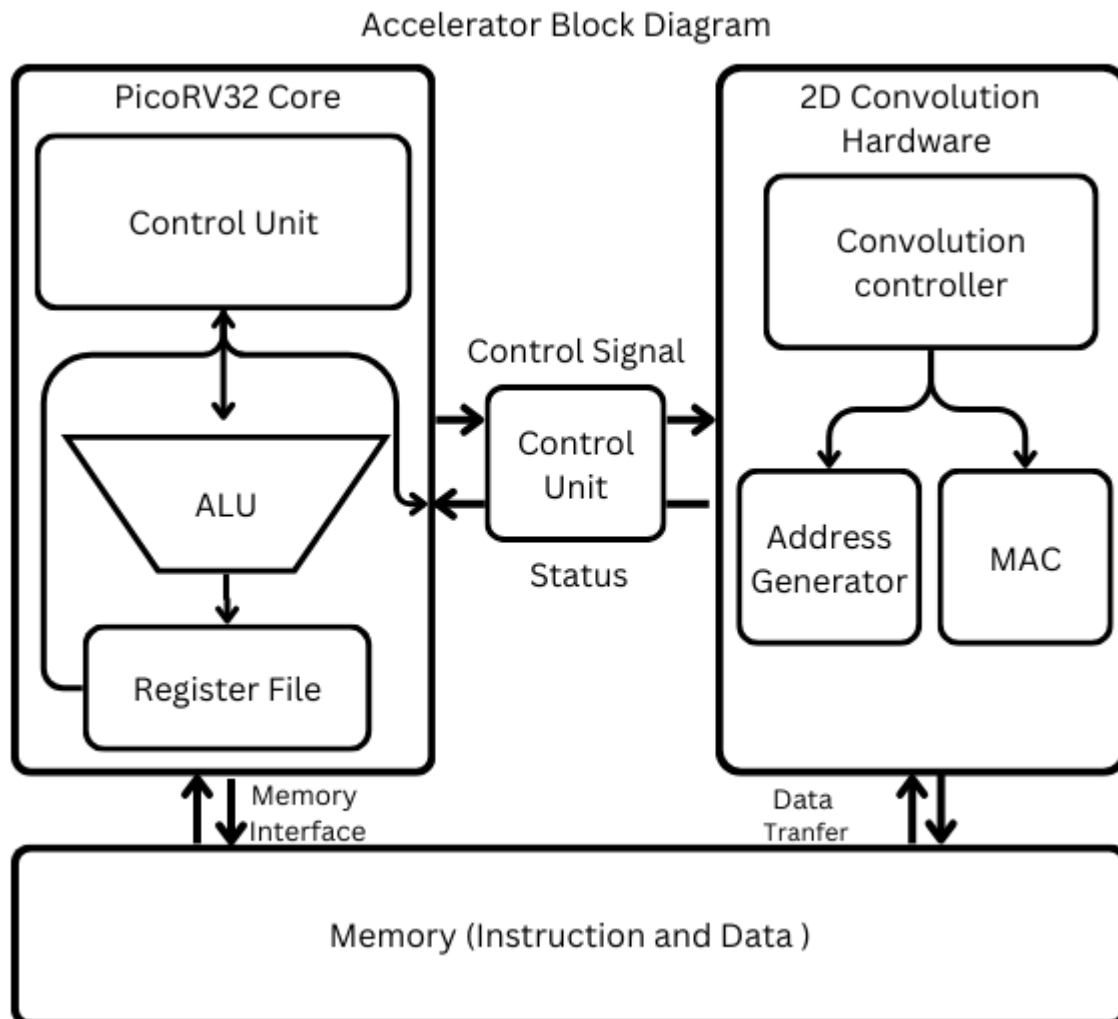
2D convolution is a fundamental operation in many applications, including image processing, computer vision, and deep learning. Optimizing the performance of this operation is critical, particularly for systems with limited computational resources, as it directly impacts the speed and efficiency of tasks such as image filtering, feature extraction, and neural network inference.

The problem involves the acceleration of 2D convolution within an RISC-V core. This process is essential to improving computational efficiency while maintaining accuracy and reliability. The Problem revolves around implementing a separate hardware block for convolution operations, while making sure that it does not affect the normal working operations, and have a net gain in terms of cycles, register usage and logic elements and to prototype a convolution accelerator that demonstrates significant speedup compared to traditional CPU-based methods.

PROPOSED SOLUTION

In our proposed solution, we advocate a separate hardware block dedicated to handling convolution operations. We will be using PicoRV32 RISC-V core as the processor and make use of pre-existing extensions like Vector extension, Multiplication & Division. By implementing pipelining and parallelism to handle the operations We will be using Fast Fourier transform convolution (FFT), which has an time complexity of $O(N \log N)$ as opposed to conventional algorithm having a complexity of $O(N^2)$,this project operates om kernel sizes 3x3,5x5,7x7. Various scenarios and operations will be emulated to ensure that there is a significant gain in performance, fewer multiplications performed, cycles saved, while not affecting the normal operation of the ALU and ensuring gain in performance, while maintaining the same power consumption.

BLOCK DIAGRAM



PROJECT EXECUTION STEPS

1. Defining Requirements: of the proposed system like the type of algorithm used, the kernel size, input matrix sizes, specifying components required for the hardware block

2. Implementation of FFT Convolution Algorithm: Implementing FFT to transform the input image for the given kernel sizes, reducing the complexity of convolution operations. Applying the inverse FFT to convert the convolved result back to the time domain. Designing MAC (Multiply Accumulate) unit to handle point-wise multiplication and accumulation.

3. Parallelism and Pipelining: Designing the FFT block with a pipelined architecture to ensure multiple stages of FFT computation can execute in parallel, reducing bottlenecks. Implementing parallel processing using vector instructions to perform FFT on multiple data points simultaneously, optimizing performance.

4. Vector Extension Implementations: Custom Vector Instructions to Define custom vector instructions (Load, Store, FFT, MAC operations) and incorporating these instructions into the PicoRV32 architecture,

5. Memory Management and Storage: enabling efficient data transfer between the FFT stages and reducing latency and implementing a mechanism in the MMU to store intermediate results during FFT processing, minimizing unnecessary memory reads/writes.

6. Simulation and Result Comparison: The Developed model will be simulated in Model sim altera with an suitable testbench and also the results will be compared with the regular computation method ,indicating the advantage obtained.

REFERENCES

- [1] S. R. Gupta, N. Papadopoulou, and M. Pericàs, "Challenges and Opportunities in the Co-design of Convolutions and RISC-V Vector Processors," in Proceedings of the SC '23 Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis (SC-W '23), New York, NY, USA: Association for Computing Machinery, 2023, pp. 1550–1556. doi: 10.1145/3624062.3624232.
- [2] K. Pavel and S. Davi, ‘Algorithms for Efficient Computation of Convolution’, Design and Architectures for Digital Signal Processing. InTech, Jan. 16, 2013. doi: 10.5772/51942.
- [3] M. Casabella, S. Sulca, I. Vignolles, and A. Pola, “Dynamic reuse of memory in 2D convolution applied to image processing,” *CASE*, pp. 145–150, Aug. 2018.
- [4] A. Lavin and S. Gray, "Fast Algorithms for Convolutional Neural Networks," 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Las Vegas, NV, USA, 2016, pp. 4013-4021, doi: 10.1109/CVPR.2016.435. keywords: {Convolution;Complexity theory;Transforms;Neural networks;Laplace equations;Two dimensional displays;Image recognition},
- [5] V. Kelefouras and G. Keramidas, "Design and Implementation of 2D Convolution on x86/x64 Processors," in IEEE Transactions on Parallel and Distributed Systems, vol. 33, no. 12, pp. 3800-3815, 1 Dec. 2022, doi: 10.1109/TPDS.2022.3171471. keywords:{Convolution;Kernel;Libraries;Optimization;Codes;Registers;Quantization(signal);Convolution;gaussianblur;code,optimization;vectorization;AVX;OpenMP;OpenCV;Intel MKL;Intel IPP;high performance computing (HPC);image processing},

GUIDE RECOMMENDATIONS:

Guide Name: Dr.Jeeru Dinesh Reddy

Designation: Assistant Professor



Signature

REMARK FROM REVIEWER:

Reviewer Name: Dr. Kiran Bailey

Designation: Assistant Professor



Signature

Reviewer Name: Dr. Archana H R

Designation: Assistant Professor



Signature